

转正答辩述职报告

• 述职人： 汪子奇

日期： 2022/1/4

议程

- 个人简介
- 试用期目标达成进展
- 重要工作总结
- 思考和总结
- 后续计划和展望

个人简介

姓名	出生年月	级别	直接主管	入职时间
汪子奇	1997/06	待评定	李琦	2022/07/13
所负责业务 (在百度)	端侧推理			
教育经历 (学校 专业 时间)	2019/09 ~ 2022/06 中国科学院大学 计算机系统结构 2014/09 ~ 2018/06 华中科技大学 计算机科学与技术			
过往工作经历 (最近1~2家)	无			

试用期目标达成进展

- 目标一：掌握日常工作流程和开发规范
 - ✓ 学习百度文化，通过《百度安全规范》考核。
 - ✓ 熟练掌握 OP 开发流程和编程规范。
- 目标二：模型性能分析与优化
 - ✓ 掌握模型性能分析工具。
 - ✓ 为Q3&Q4 Benchmark模型库，增加5个Good模型以及7个Same模型，减少12个Bad模型。
 - ✓ 新增4个算子，优化10+个kernel。
- 目标三：积极参与值班任务，保证生态用户体验，积极参与技术分享。
 - ✓ 参与github及微信群日常值班任务。
 - ✓ 参与两次技术分享，部门知识库产出三篇技术文档。

试用期总结



重要工作总结

1. 端侧推理常用优化手段与实践
2. FP16后端卷积选择策略Benchmark
3. FP16后端GEMM优化与技术分享

1. 端侧推理常用优化手段与实践

- 工作背景

方向	模型	优化前
检测	PPYOLOE-I	MNN失败
	YOLOv5s	-24.33%
	YOLOv6s	-37.48%
	YOLOv7	-45.99%
	PicoDet	-1.99%
	TinyPose_128x96	-21.39%
	TinyPose_256x192	-10.04%
分类	ResNet50_vd	-24.20%
	MobileNetV3_large_x1_0	39.59%
	PPLCNetV2	1.22%
	PPHGNet_tiny	-7.50%
	EfficientNetB0	-23.92%
分割	PP-Liteseg	MNN失败
	PP-HumanSeg-Lite	20.61%
	HRNet	-14.87%
	UNet	-14.47%
	Deeplabv3-ResNet50	-13.68%

GOOD	SAME	BAD
($\geq 10\%$)	(-10%, 10%)	($\leq -10\%$)
4	3	10

Q4 Lite Benchmark中选取的业界有代表性的模型，性能落后于竞品。

1.端侧推理常用优化手段与实践

- 目标：对Lite Benchmark中选取的业界有代表性的模型，要求其性能超过或追平竞品。
- 进展：检测、分类模型达到预设目标，分割模型仍有微弱差距。

方向	模型	优化前	优化后
检测	PPYOLOE-I	MNN失败	MNN失败
	YOLOv5s	-24.33%	23.50%
	YOLOv6s	-37.48%	-9.18%
	YOLOv7	-45.99%	11.06%
	PicoDet	-1.99%	-1.99%
	TinyPose_128x96	-21.39%	-7.39%
	TinyPose_256x192	-10.04%	2.47%
分类	ResNet50_vd	-24.20%	-5.48%
	MobileNetV3_large_x1_0	39.59%	39.59%
	PPLCNetV2	1.22%	1.22%
	PPHNet_tiny	-7.50%	-7.50%
	EfficientNetB0	-23.92%	27.89%
分割	PP-LiteSeg	MNN失败	MNN失败
	PP-HumanSeg-Lite	20.61%	20.61%
	HRNet	-14.87%	-14.87%
	UNet	-14.47%	-10.80%
	Deeplabv3-ResNet50	-13.68%	-10.85%

GOOD	SAME	BAD
($\geq 10\%$)	(-10%, 10%)	($\leq -10\%$)
4	3	10



GOOD	SAME	BAD
($\geq 10\%$)	(-10%, 10%)	($\leq -10\%$)
6	7	3

端侧推理常用优化手段1

- 算子融合优化

算子是神经网络中的最小调度单位，但遗憾的是，它并不是原子的：一个复杂的算子可以被更细粒度的算子所表示（网络中应该避免使用特殊算子）：



Matmul
<pre>for (int i = 0; i < M; i++) { for (int j = 0; j < N; j++) { for (int k = 0; k < K; k++) { out[i][j] = in[i][k] * weights[k][j]; } } }</pre>

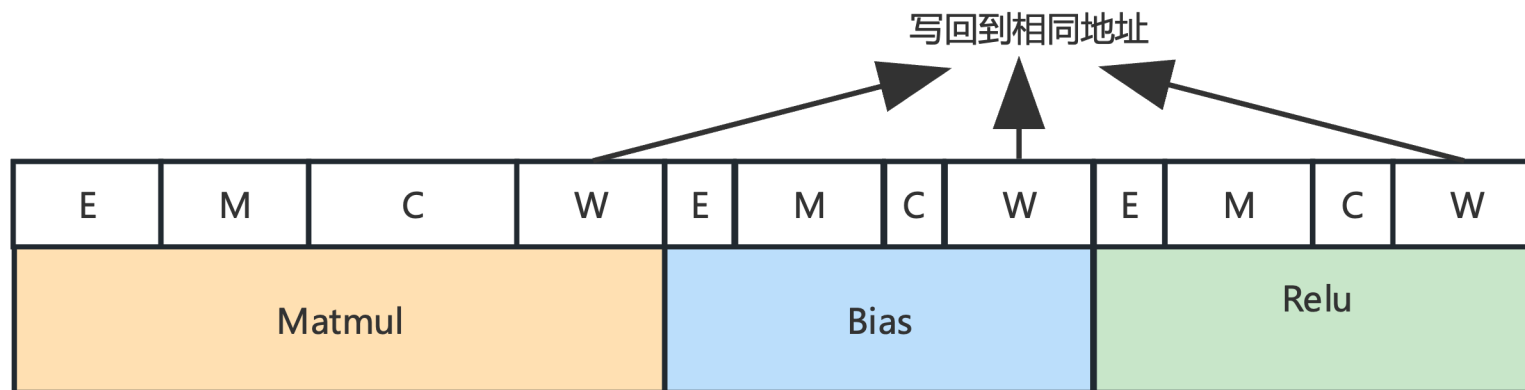
Bias Add
<pre>for (int i = 0; i < M; i++) { for (int j = 0; j < N; j++) { out[i][j] += bias[i]; } }</pre>

Relu
<pre>for (int i = 0; i < M; i++) { for (int j = 0; j < N; j++) { out[i][j] = in[i][j] * (in[i][j] > 0); } }</pre>

- 在Matmul+Bias+Relu的子网中，如果不融合算子，out将至少被读取3次，并且还附有启动3个算子的时间。

端侧推理常用优化手段1

- 算子执行“流水线”

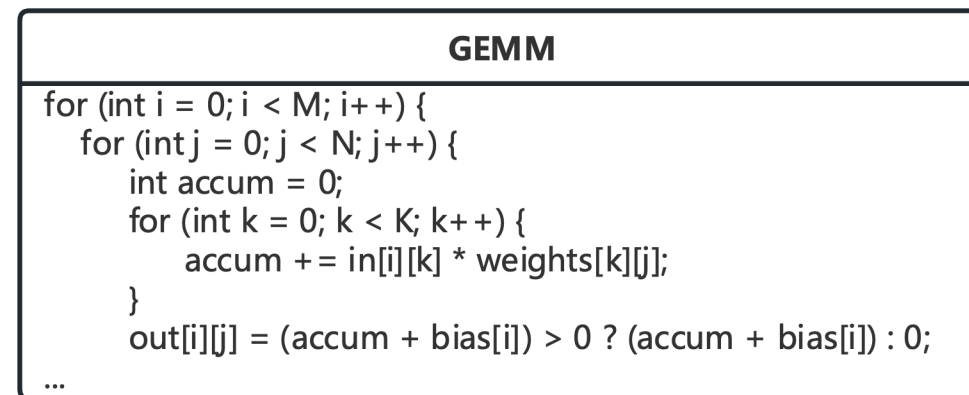
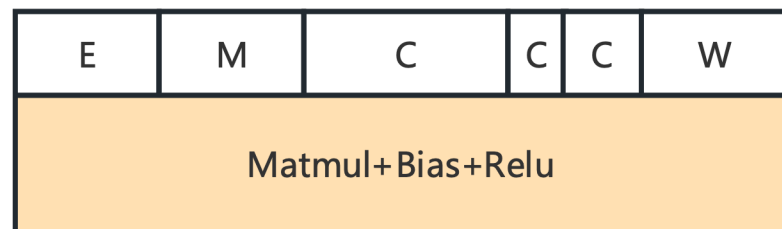
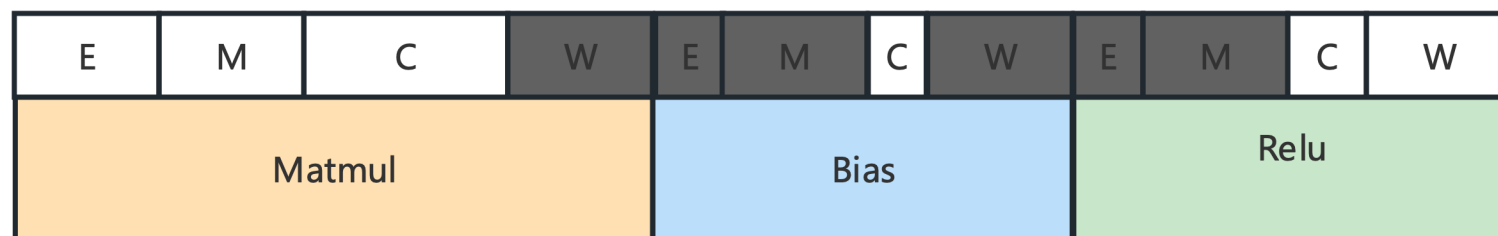


E : Op创建
M : 参数访存
C : Kernel计算
W : 参数写回

- 对于Matmul这类计算量大的Op而言，其各个阶段的耗时都偏大。
- 对于Bias和激活这类计算量小的Op而言，性能瓶颈在参数的访存上。

端侧推理常用优化手段1

- 算子融合优化后 “流水线”

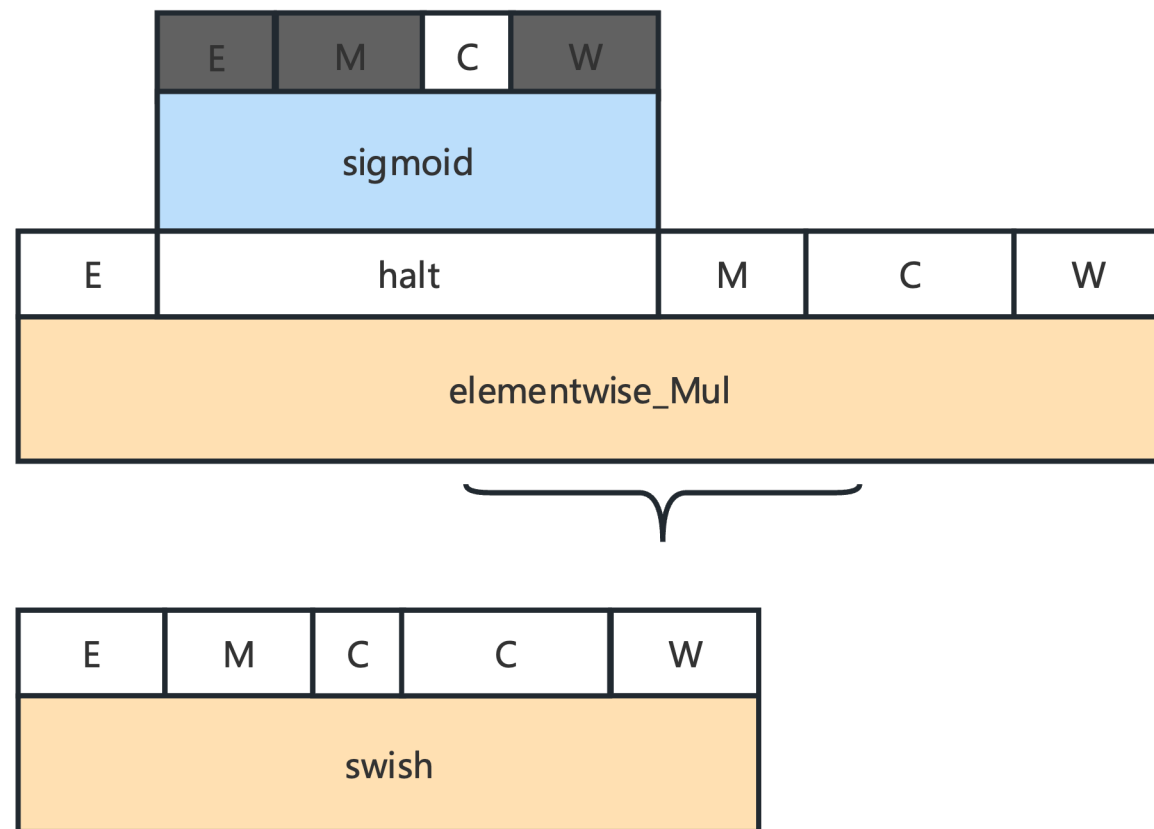
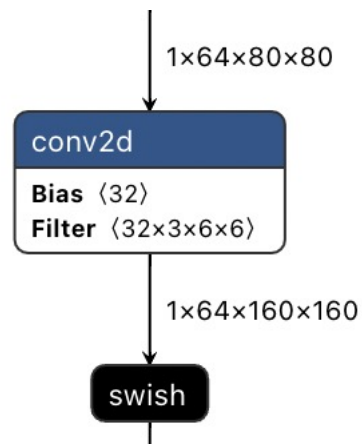
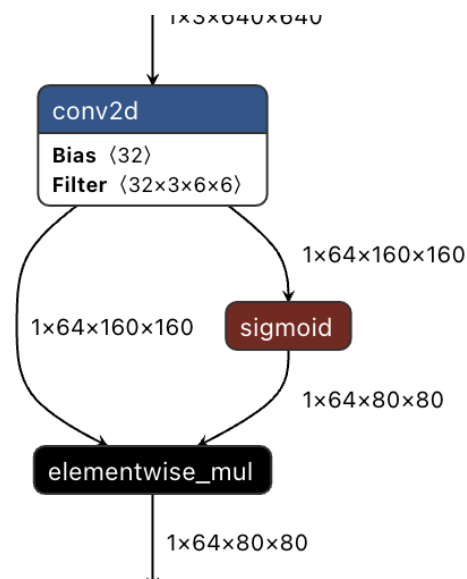


- 图融合减少访存，以及算子Overhead。

Yolo系列模型优化实战

- 常见网络结构融合优化

➤ $\text{swish} = x * \text{sigmoid}(\text{Beta} * x)$



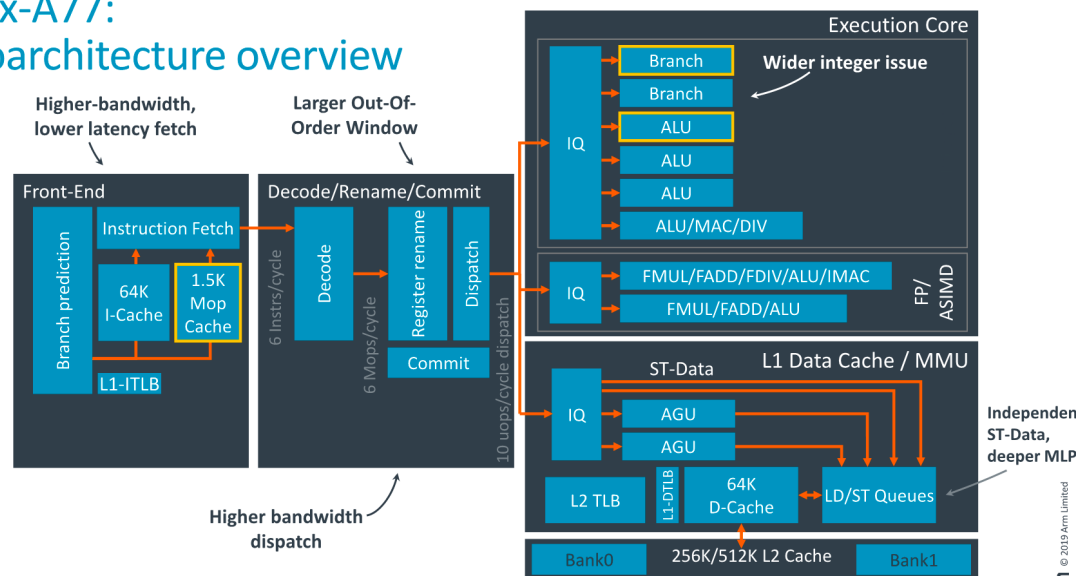
端侧推理常用优化手段2

- 基于CPU微架构的算子实现优化

- 现代CPU，一个物理核通常有多个向量计算单元，充分利用硬件计算资源可以有效地提升计算效率。

```
151 - for (int k = 0; k < neon_loop_cnt_dim8; ++k) {
152 -     float16x8_t v_x = vld1q_f16(ptr_in_thread);
153 -     float16x8_t v_exp = expq_ps_f16(vnegq_f16(vmulq_f16(vbeta, v_x)));
154 -     v_x = vdivq_f16(v_x, vaddq_f16(vone, v_exp));
155 -     vst1q_f16(ptr_out_thread, v_x);
156 -     ptr_out_thread += 8;
157
147 + float16x8_t vbeta = vdupq_n_f16(beta);
148 + float16x8_t vone = vdupq_n_f16(1.f16);
149 + int i = 0;
150 + for (; i + 31 < nums_per_thread; i += 32) {
151 +     float16x8_t v_p0 = vld1q_f16(ptr_in_thread);
152 +     float16x8_t v_p1 = vld1q_f16(ptr_in_thread + 8);
153 +     float16x8_t v_p2 = vld1q_f16(ptr_in_thread + 16);
154 +     float16x8_t v_p3 = vld1q_f16(ptr_in_thread + 24);
155 +     v_p0 = vdivq_f16(v_p0, vaddq_f16(vone, exp_ps(vnegq_f16(vmulq_f16(vbeta, v_p0)))));
156 +     v_p1 = vdivq_f16(v_p1, vaddq_f16(vone, exp_ps(vnegq_f16(vmulq_f16(vbeta, v_p1)))));
157 +     v_p2 = vdivq_f16(v_p2, vaddq_f16(vone, exp_ps(vnegq_f16(vmulq_f16(vbeta, v_p2)))));
158 +     v_p3 = vdivq_f16(v_p3, vaddq_f16(vone, exp_ps(vnegq_f16(vmulq_f16(vbeta, v_p3)))));
```

Cortex-A77: Microarchitecture overview

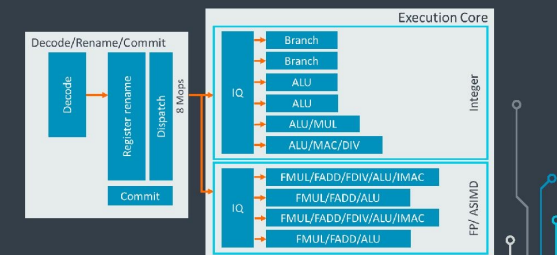


The embargo for this content presented at Arm Tech Day will lift on Sunday, May 26 at 9:00 p.m. PT. Corresponding UK and Taiwan times are: Monday, May 27 at 5:00 a.m. BST / Monday, May 27 at 12:00 p.m. China Standard Time

Arm Cortex-X2: Out-of-Order Core

Optimizing depth and latency

- Remove a pipeline stage at dispatch
 - Now an overall 10-cycle pipeline
- Out of order window size growth
 - Up to 30%
 - ROB now 288+ entries
- SVE 128b vector length implementation
- Up to 2x ML performance
 - bfloat16, int8 matmul data types
- Additional instruction fusion cases



arm
© 2021 Arm Limited

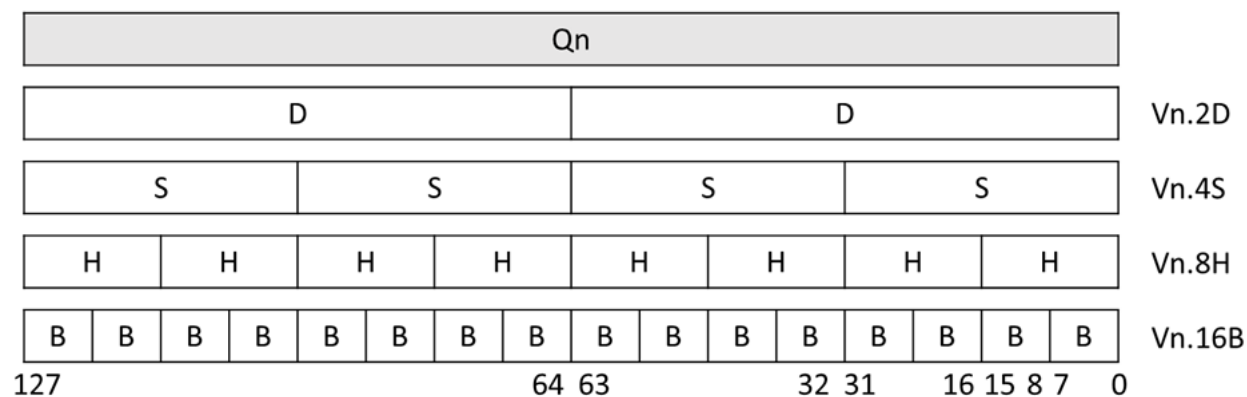
© 2021 Arm Limited

This content is under strict embargo until Tuesday, May 25 @ 6:00 a.m. PT. Corresponding UK and China times are 2 p.m. BST and 9 p.m. CST (China Standard Time), respectively.

arm

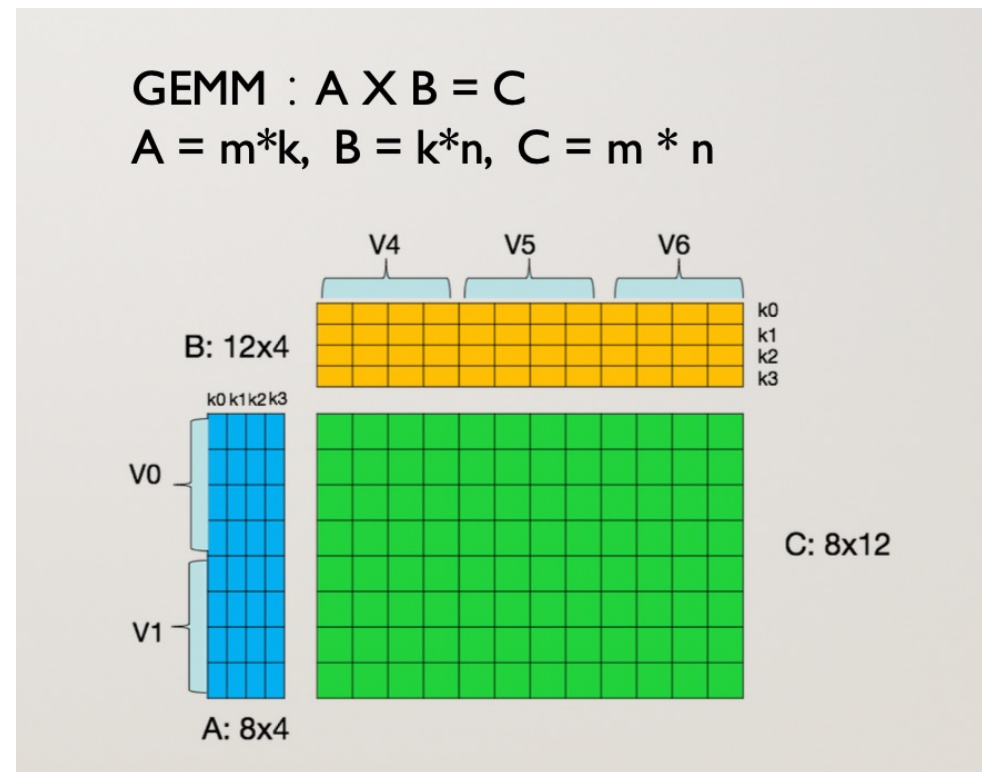
- 基于CPU微架构的算子实现优化

- ARM v8 --- 32个V寄存器 (128bits)



端侧推理常用优化手段2

- 对于不同指令集而言，逻辑寄存器的数目影响了kernel的具体设计思路---以GEMM为例
 - ARM v8
 - 当 $m > 8$ 时，micro kernel为 8×12
 - ARM v7
 - 当 $m > 4$ 时，micro kernel为 4×8
- 寄存器使用分析（以v8为例）：
 - 输出cblock块大小为 8×12 ，将占用 8×3 ，共计24个neon寄存器。
 - 输入ablock块大小为 8×4 ，在k（列）方向上按4展开，读取一行数据需要占用2个neon寄存器。
 - 输入bblock块大小为 4×12 ，在k（行）方向上按4展开，读取一行数据需要占用3个neon寄存器。
 - 剩下的寄存器用作数据预取。



端侧推理常用优化手段3

- 汇编实现的优化

- 数据冲突：减少数据依赖冲突
- 资源冲突：提高每个处理器的利用率
- 充分利用硬件流水线，手动调整代码顺序以减少停顿周期

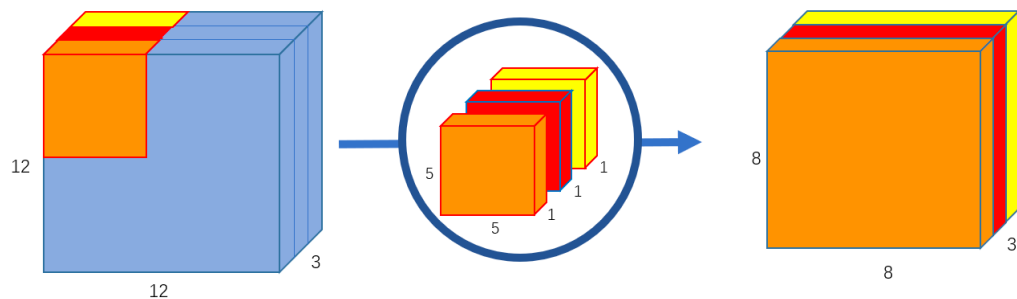


由于硬件的乱序执行和指令调度机制，实践中，对汇编代码进行的优化往往很难符合理论预期。

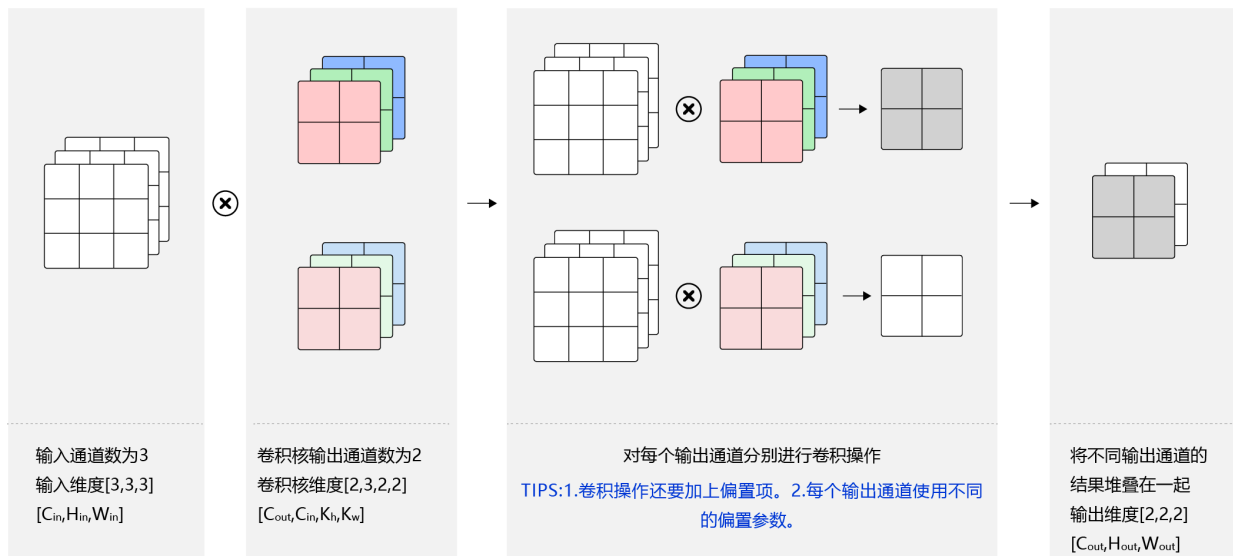
2.FP16后端卷积选择策略Benchmark

• 工作背景

- 卷积算子的实现原理在不同的推理引擎中大同小异，但在实际优化时，模型性能从持平到领先的瓶颈经常就是在卷积的实现上。
- 不同模型中的卷积参数差别巨大，用单测难以对卷积的性能有清晰的认知。
- 当前卷积选择策略存在判断条件冗余、无依据的情况，且缺少特定fastpath实现和通用实现GEMM间的性能对比。



- 输出通道的数目通常也被称作卷积核的个数，这里有两个卷积核。
- 红绿蓝代表第一个卷积核的三个输入通道；浅红浅绿浅蓝代表第二个卷积核的三个输入通道。



2.FP16后端卷积选择策略Benchmark

- 解决方案

- 通过google benchmark工具梳理不同shape下的Lite fp16端卷积实现性能。

- 效果

~ (ssh)	第1	lite-7820 (ssh)	第2	lite-7820 (ssh)	第3	root@paddle-precision-7820-tower: ~/Mytest/glog...	第4
f16_conv/Depth3s1/N:1/H:224/W:224/G:1/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:1/SW:1/DH:1/DW:1/real_time	16382	ns	16335	ns	36178	OPS=55.1324G/s	
f16_conv/Depth3s1/N:1/H:224/W:224/G:2/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:1/SW:1/DH:1/DW:1/real_time	32225	ns	32166	ns	21743	OPS=56.0547G/s	
f16_conv/Depth3s1/N:1/H:224/W:224/G:4/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:1/SW:1/DH:1/DW:1/real_time	64025	ns	63887	ns	10930	OPS=56.4257G/s	
f16_conv/Depth3s1/N:1/H:224/W:224/G:8/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:1/SW:1/DH:1/DW:1/real_time	127476	ns	127248	ns	5492	OPS=56.6801G/s	
f16_conv/Depth3s1/N:1/H:224/W:224/G:16/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:1/SW:1/DH:1/DW:1/real_time	255697	ns	255106	ns	2717	OPS=56.5149G/s	
f16_conv/Depth3s1/N:1/H:256/W:192/G:1/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:1/SW:1/DH:1/DW:1/real_time	16012	ns	15985	ns	43653	OPS=55.2561G/s	
f16_conv/Depth3s1/N:1/H:256/W:192/G:2/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:1/SW:1/DH:1/DW:1/real_time	31516	ns	31463	ns	22188	OPS=56.1446G/s	
f16_conv/Depth3s1/N:1/H:256/W:192/G:4/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:1/SW:1/DH:1/DW:1/real_time	62571	ns	62433	ns	11175	OPS=56.5589G/s	
f16_conv/Depth3s1/N:1/H:256/W:192/G:8/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:1/SW:1/DH:1/DW:1/real_time	124602	ns	124361	ns	5437	OPS=56.8039G/s	
f16_conv/Depth3s1/N:1/H:256/W:192/G:16/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:1/SW:1/DH:1/DW:1/real_time	249489	ns	248938	ns	2779	OPS=56.7391G/s	
f16_conv/Depth3s1/N:1/H:320/W:320/G:1/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:1/SW:1/DH:1/DW:1/real_time	32635	ns	32577	ns	21431	OPS=56.4788G/s	
f16_conv/Depth3s1/N:1/H:320/W:320/G:2/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:1/SW:1/DH:1/DW:1/real_time	65031	ns	64910	ns	10745	OPS=56.6872G/s	
f16_conv/Depth3s1/N:1/H:320/W:320/G:4/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:1/SW:1/DH:1/DW:1/real_time	129604	ns	129365	ns	5401	OPS=56.8873G/s	
f16_conv/Depth3s1/N:1/H:320/W:320/G:8/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:1/SW:1/DH:1/DW:1/real_time	260264	ns	259673	ns	2664	OPS=56.6564G/s	
f16_conv/Depth3s1/N:1/H:320/W:320/G:16/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:1/SW:1/DH:1/DW:1/real_time	597384	ns	595342	ns	1179	OPS=49.3673G/s	
f16_conv/Depth3s1/N:1/H:488/W:488/G:1/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:1/SW:1/DH:1/DW:1/real_time	75586	ns	75425	ns	9236	OPS=56.7115G/s	
f16_conv/Depth3s1/N:1/H:488/W:488/G:2/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:1/SW:1/DH:1/DW:1/real_time	152113	ns	151838	ns	4596	OPS=56.3605G/s	
f16_conv/Depth3s1/N:1/H:488/W:488/G:4/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:1/SW:1/DH:1/DW:1/real_time	307364	ns	306066	ns	2281	OPS=55.7852G/s	
f16_conv/Depth3s1/N:1/H:488/W:488/G:8/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:1/SW:1/DH:1/DW:1/real_time	696528	ns	694337	ns	984	OPS=49.2338G/s	
f16_conv/Depth3s1/N:1/H:488/W:488/G:16/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:1/SW:1/DH:1/DW:1/real_time	1545468	ns	1537839	ns	423	OPS=44.3785G/s	
f16_conv/Depth3s1/N:1/H:640/W:640/G:1/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:1/SW:1/DH:1/DW:1/real_time	133920	ns	133618	ns	5363	OPS=55.0538G/s	
f16_conv/Depth3s1/N:1/H:640/W:640/G:2/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:1/SW:1/DH:1/DW:1/real_time	264509	ns	263943	ns	2631	OPS=55.7471G/s	
f16_conv/Depth3s1/N:1/H:640/W:640/G:4/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:1/SW:1/DH:1/DW:1/real_time	587915	ns	586220	ns	1129	OPS=50.1624G/s	
f16_conv/Depth3s1/N:1/H:640/W:640/G:8/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:1/SW:1/DH:1/DW:1/real_time	1259701	ns	1254072	ns	531	OPS=46.8256G/s	
f16_conv/Depth3s1/N:1/H:640/W:640/G:16/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:1/SW:1/DH:1/DW:1/real_time	2798635	ns	2782286	ns	247	OPS=42.1508G/s	
f16_conv/Depth3s2/N:1/H:224/W:224/G:1/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:2/SW:2/DH:1/DW:1/real_time	7953	ns	7930	ns	87976	OPS=28.3913G/s	
f16_conv/Depth3s2/N:1/H:224/W:224/G:2/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:2/SW:2/DH:1/DW:1/real_time	15316	ns	15271	ns	45514	OPS=29.484G/s	
f16_conv/Depth3s2/N:1/H:224/W:224/G:4/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:2/SW:2/DH:1/DW:1/real_time	30444	ns	30392	ns	22951	OPS=29.6667G/s	
f16_conv/Depth3s2/N:1/H:224/W:224/G:8/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:2/SW:2/DH:1/DW:1/real_time	61835	ns	61726	ns	10805	OPS=29.2124G/s	
f16_conv/Depth3s2/N:1/H:224/W:224/G:16/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:2/SW:2/DH:1/DW:1/real_time	138564	ns	138309	ns	5226	OPS=26.0722G/s	
f16_conv/Depth3s2/N:1/H:256/W:192/G:1/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:2/SW:2/DH:1/DW:1/real_time	7630	ns	7611	ns	92158	OPS=28.9893G/s	
f16_conv/Depth3s2/N:1/H:256/W:192/G:2/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:2/SW:2/DH:1/DW:1/real_time	14476	ns	14433	ns	47689	OPS=30.5587G/s	
f16_conv/Depth3s2/N:1/H:256/W:192/G:4/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:2/SW:2/DH:1/DW:1/real_time	28851	ns	28803	ns	24556	OPS=30.6662G/s	
f16_conv/Depth3s2/N:1/H:256/W:192/G:8/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:2/SW:2/DH:1/DW:1/real_time	60944	ns	60831	ns	12315	OPS=29.0343G/s	
f16_conv/Depth3s2/N:1/H:256/W:192/G:16/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:2/SW:2/DH:1/DW:1/real_time	123416	ns	123123	ns	5628	OPS=28.6748G/s	
f16_conv/Depth3s2/N:1/H:320/W:320/G:1/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:2/SW:2/DH:1/DW:1/real_time	15655	ns	15611	ns	44243	OPS=29.4354G/s	
f16_conv/Depth3s2/N:1/H:320/W:320/G:2/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:2/SW:2/DH:1/DW:1/real_time	31040	ns	30988	ns	22355	OPS=29.6906G/s	
f16_conv/Depth3s2/N:1/H:320/W:320/G:4/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:2/SW:2/DH:1/DW:1/real_time	62904	ns	62794	ns	11141	OPS=29.3016G/s	
f16_conv/Depth3s2/N:1/H:320/W:320/G:8/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:2/SW:2/DH:1/DW:1/real_time	141395	ns	141127	ns	4969	OPS=26.0717G/s	
f16_conv/Depth3s2/N:1/H:320/W:320/G:16/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:2/SW:2/DH:1/DW:1/real_time	292404	ns	291808	ns	2244	OPS=25.2145G/s	
f16_conv/Depth3s2/N:1/H:488/W:488/G:1/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:2/SW:2/DH:1/DW:1/real_time	36986	ns	36915	ns	18944	OPS=28.9747G/s	
f16_conv/Depth3s2/N:1/H:488/W:488/G:2/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:2/SW:2/DH:1/DW:1/real_time	74699	ns	74561	ns	9022	OPS=28.6926G/s	
f16_conv/Depth3s2/N:1/H:488/W:488/G:4/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:2/SW:2/DH:1/DW:1/real_time	149726	ns	149444	ns	4632	OPS=28.6296G/s	
f16_conv/Depth3s2/N:1/H:488/W:488/G:8/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:2/SW:2/DH:1/DW:1/real_time	333263	ns	332286	ns	2064	OPS=25.7256G/s	
f16_conv/Depth3s2/N:1/H:488/W:488/G:16/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:2/SW:2/DH:1/DW:1/real_time	766279	ns	764415	ns	853	OPS=22.3762G/s	
f16_conv/Depth3s2/N:1/H:640/W:640/G:1/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:2/SW:2/DH:1/DW:1/real_time	65994	ns	65870	ns	10636	OPS=27.9299G/s	
f16_conv/Depth3s2/N:1/H:640/W:640/G:2/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:2/SW:2/DH:1/DW:1/real_time	144661	ns	144326	ns	5091	OPS=25.4831G/s	
f16_conv/Depth3s2/N:1/H:640/W:640/G:4/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:2/SW:2/DH:1/DW:1/real_time	299718	ns	299046	ns	2316	OPS=24.5991G/s	
f16_conv/Depth3s2/N:1/H:640/W:640/G:8/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:2/SW:2/DH:1/DW:1/real_time	643076	ns	641063	ns	1063	OPS=22.9298G/s	
f16_conv/Depth3s2/N:1/H:640/W:640/G:16/GCin:1/KH:3/KW:3/PU:1/PO:1/PL:1/PR:1/SH:2/SW:2/DH:1/DW:1/real_time	1356710	ns	1350862	ns	511	OPS=21.7373G/s	

- 结论：

- depthwise kernel在5x5 Groups 性能在ic以及oc小的情况下远低于3x3。
- depthwise common现状与depthwise kernel 5x5类似。
- depthwise conv 整体性能大幅优于gemm，接近10x。
- Direct kernel在ic和oc较小时是GEMM的1.5~2x。但更大时，两者几乎无差别。

3.FP16后端GEMM优化与技术分享

- 工作背景

- 受昆仑芯平台软件研发部的【XTCL知识分享】组织者的邀请，做关于Paddle Lite在ARM端的Cache优化技术分享。[知识库文档](#)

[已过期] XTCL知识分享：PaddleLite 在Conv和GEMM利用cache分块优化

2022年 09月28日 周三 14:00 – 15:00 视频会议

创建纪要 创建会话 智能记录 复制会议链接 进入会议

3人接受 0人拒绝 2人待定 34人尚未回复 0人无权限 39人 >

组织者
臧战

Ma Yin

王垒

师紧想

刘彧

胡文明

顾凯栋

赵益萱

任鸽

周佳敏

李欢

张钊

黎权锋

曾浩伦

汪子奇

liyan8...

+ 添加知识库文档 + 添加本地文档

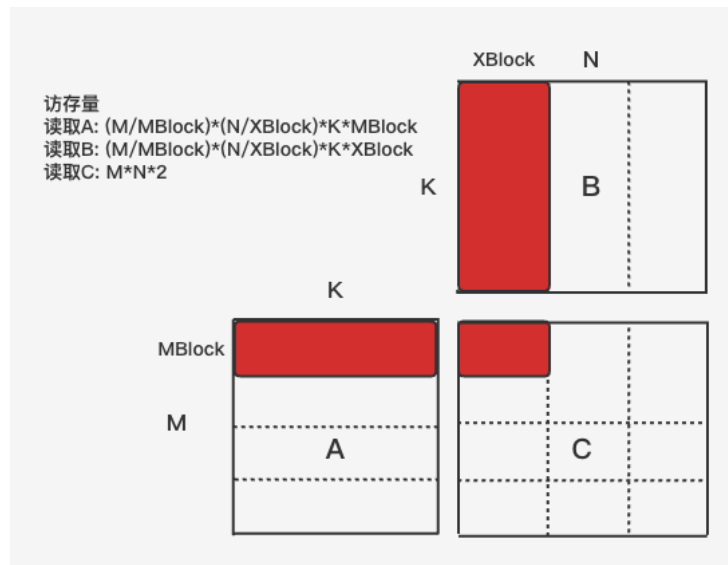
Gemm-矩阵分块与寄存器优化

+ 添加会议描述

每周周一重复

3.FP16后端GEMM优化与技术分享

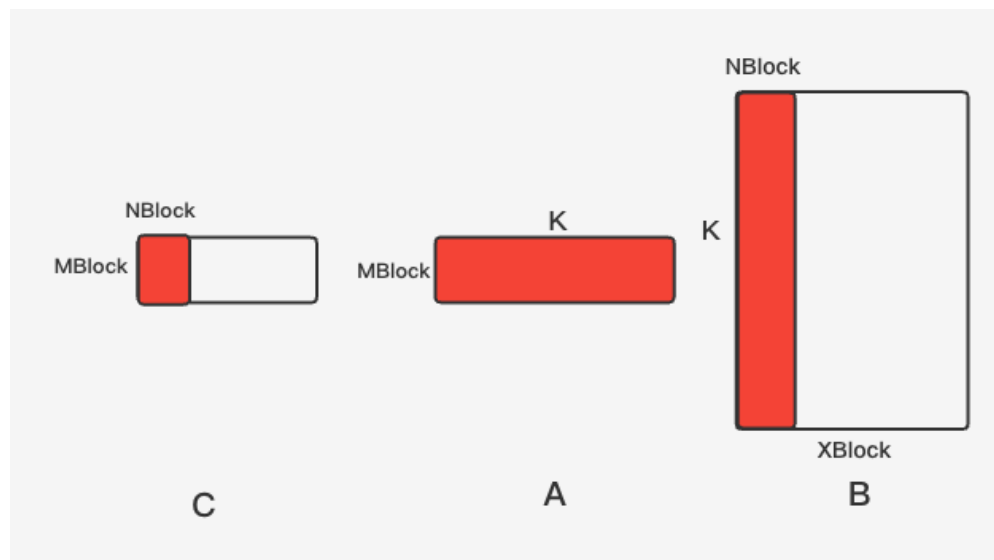
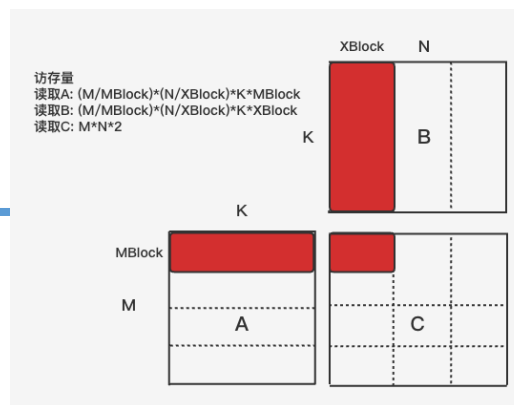
- 矩阵分块策略
 - 为了减少Gemm计算时的访存开销，最有效的方法是对Gemm的计算进行分块，并将分块之后的数据保存在CPU的Cache中，充分利用Cache提高访存效率。
 - 将A矩阵按照MBlock进行行分块，将B矩阵按照XBlock进行列分块，计算核大小为MBlock x NBlock。
 - GEMM的最外层循环按XBlock维度展开，因此将B的列块尽可能存放在L2中，可以使每次读取B的一个列块都是从L2中读取，访存代价最小。



3.FP16后端GEMM优化与技术分享

- 计算核分块策略

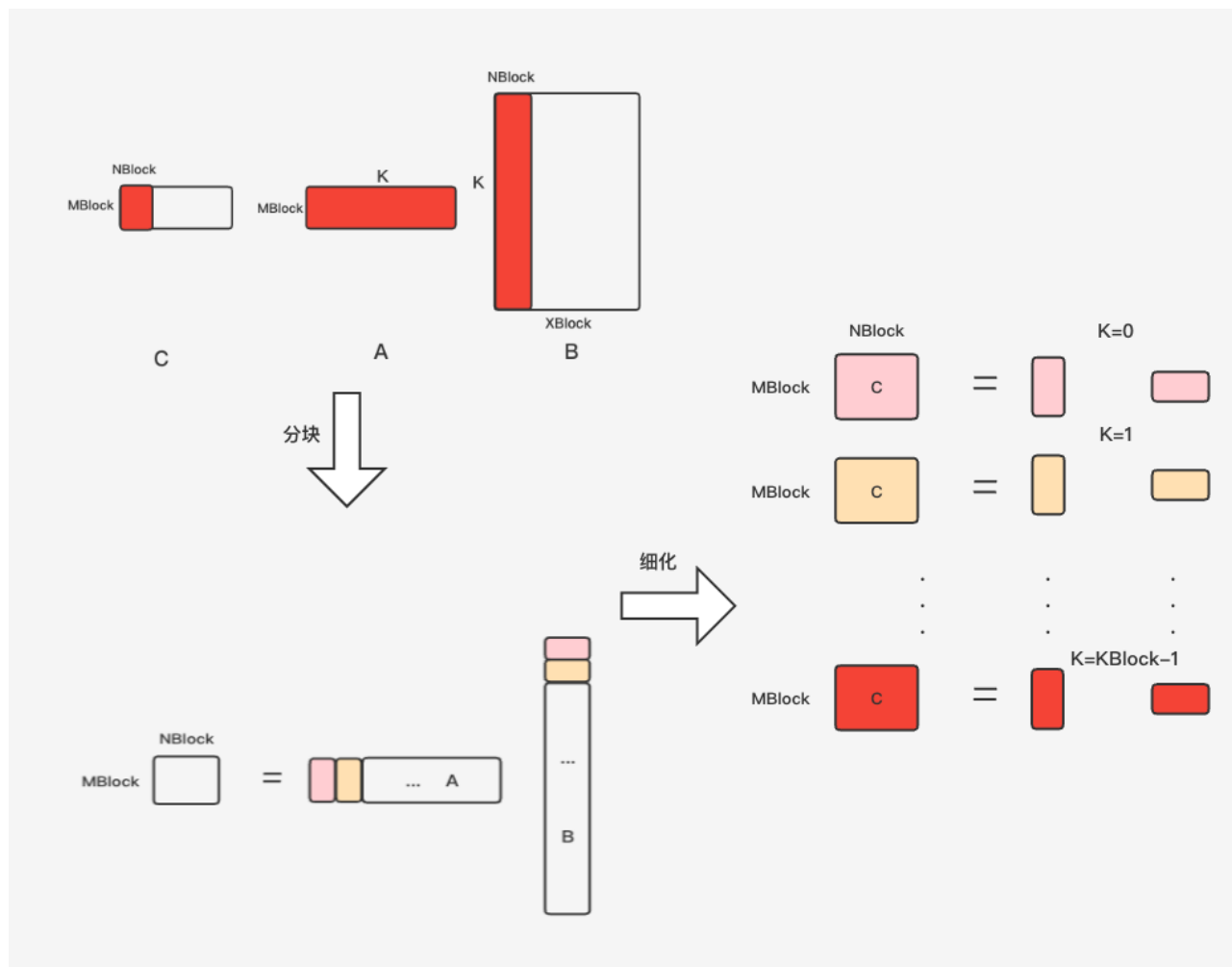
- 尽可能将图中红色部分（计算核对应的输入矩阵数据）保存在L1D中，以确保最内层循环中发生的计算不受访存带宽的限制。
- 当K较大时，只能保证KBlock(< K)对应的计算发生在L1 Cache中。
- 根据以下算法计算出不同部署环境（CPU）下Gemm计算的分块大小：
 1. 根据CPU处理器的寄存器数量得到MBlock和NBlock的具体大小，寄存器容量 > MBlock*NBlock*sizeof(elem)。
 2. 根据L1D Cache的大小结合MBlock和NBlock计算出KBlock， $KBlock = L1D / (MBlock + NBlock)$ 。
 3. 最后根据L2 Cache的大小计算出B矩阵的列块大小XBlock， $XBlock = (L2 - L1D) / K$ 。



3.FP16后端GEMM优化与技术分享

- 计算核

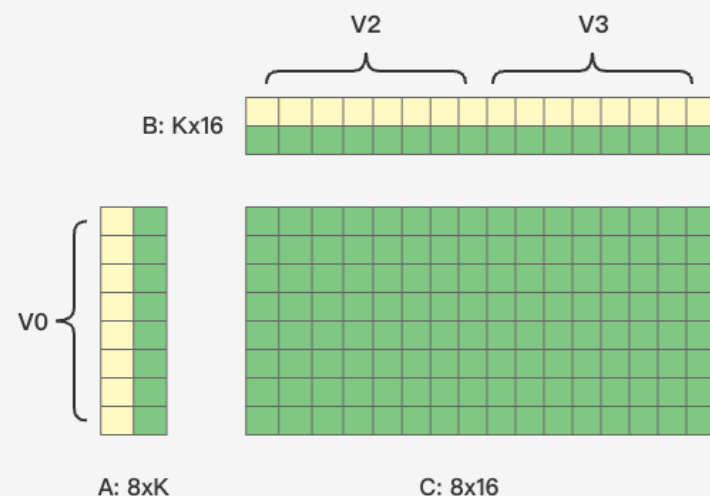
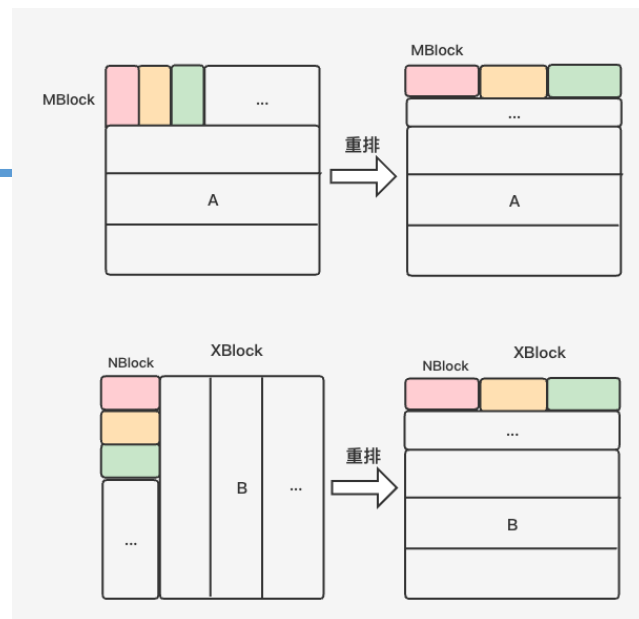
- 计算核发生的场所在L1 Cache中，访问效率最高，能充分利用硬件计算资源。
- 在K维度上，会读取A中一列数据和B中一行数据进行矩阵外积算法，得到大小为MBlock*NBlock的C，如此循环K/KBlock次，完成最内层循环的计算。



3.FP16后端GEMM优化与技术分享

- 内存重排与SIMD

- 处理器按cache line取数据，为了尽可能提高cache的命中率，仍需对数据进行重排操作以提高实际性能。
- 仅需3条ldr指令就可满足 $8*16*2=512$ 次乘法操作所需的操作数，重排操作为使用向量寄存器提供了条件，进而可以利用neon指令（SIMD）来对计算进行加速。



思考和总结

产品方面：

- 目前Lite端侧的用户增长强依赖于套件们的引流，端侧Demo工程对新用户的吸引力不足。

团队方面：

- 产品有技术深度，上接深度学习算法，下接部署硬件。但缺少技术说明文档，对新人不友好。
- 日常工作面向的是模型，算法工程师对于端侧模型的设计思路和取舍我们无从得知，缺少这方面的沟通，双方对于性能的认知是不同的，可能会造成有些模型优化难度大的问题。

个人方面：

- 对于Paddle主框架不熟悉，需要加强对fluid框架的学习，为明年做准备。
- 从性能优化的角度切入了推理部署的领域，未来还需要扩展自己的知识面和技术深度。
- 注重个人软技能的提升，提升团队协作沟通等能力，养成总结的习惯。
- 工作中需要更加主动的和不同组的同学进行沟通，在沟通中学习。

后续计划和展望

- 从推理模型性能优化方向切入了推理部署领域，未来仍然需要学习更多推理部署领域的进阶知识，比如端侧模型设计思路、模型压缩及量化等，道阻且长。
- 学习Paddle主框架的代码，提升对Paddle的理解。
- 学习和调研其它竞品，从外部汲取更多性能优化的技术，增加自身技术深度。
- 关注业界进展，抬起头看世界。
- 养成好的职业习惯。

谢谢大家！